Package 'RFDose'

October 19, 2026

Type Package

Title Deterministic and stochastic radiofrequency electromagnetic field (RF-EMF) dose calculator

Version 2.0.0

Description Package for calculating RF-EMF doses for cohorts. Calculates the daily RF-EMF dose of the brain and the whole body from the self-reported use of wireless devices and the environment, with SAR values from the GOLIAT dosimetric simulations. A stochastic version estimates the uncertainty of the doses by drawing the parameters, SAR values and inputs from their distributions.

License MIT + file LICENSE

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

Imports dplyr,
stats,
tidyr,
utils,
yaml

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0),
tibble

VignetteBuilder knitr

Depends R (>= 3.5)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Contents

RFDose-package 3
act_pwr_props 5
available_sar_files 5
calculate_emf_doses 6
calculate_emf_doses_stochastic 7
check_boolean_not_na 9

check_character_not_na . . . 9
check_country . . . 10
check_duration . . . 10
check_headp_num . . . 11
check_numeric_not_na . . . 11
check_proportions . . . 12
check_shape . . . 12
check_tissue . . . 13
check_urbanicity . . . 13
cordless_dose . . . 14
dect_msar . . . 15
dect_pwr . . . 16
dect_sar . . . 16
example_data . . . 17
farfield_dose . . . 18
farfield_msar . . . 19
farfield_pwr . . . 20
farfield_sar . . . 20
fill_missing_variables . . . 21
gaming_dose . . . 22
get_param . . . 23
headphones_dose . . . 23
headphones_earset_dose . . . 24
headphones_phone_dose . . . 25
hotspot_dose . . . 26
laptop_dose . . . 27
laptop_msar . . . 28
laptop_pwr . . . 29
laptop_sar . . . 30
list_distributions . . . 31
list_sar . . . 32
load_distributions . . . 32
load_input_distributions . . . 33
load_params . . . 34
load_tissue_params . . . 35
location_props . . . 35
log_pnorm_between . . . 36
mobilecall_dose . . . 36
mobiledata_dose . . . 38
mpc_bt_earset_msar . . . 40
mpc_bt_phone_msar . . . 40
mpc_msar . . . 41
mpc_pwr_data . . . 42
mpc_pwr_native . . . 43
mpc_pwr_wifi . . . 44
mpc_sar_data . . . 44
mpc_sar_native . . . 45
mpc_sar_wifi . . . 46
mpd_msar . . . 47
mpd_pwr_data . . . 49
mpd_pwr_wifi . . . 50
mpd_sar_data . . . 51

mpd_sar_wifi . . . 52
other_dose_wrapper . . . 52
prepare_dose_data . . . 54
q_around . . . 54
q_beta . . . 55
q_category . . . 55
q_shares . . . 56
q_trunc_lognormal . . . 56
r_beta . . . 57
r_dirichlet . . . 57
r_lognormal . . . 58
r_trunc_lognormal . . . 58
sar_file_path . . . 59
set_params . . . 59
simulate_inputs . . . 60
simulate_params . . . 60
smartwatch_dose . . . 61
smartwatch_phone_dose . . . 62
smartwatch_watch_dose . . . 63
stochastic_doses . . . 64
summarise_doses . . . 64
tablet_dose . . . 65
tablet_msar . . . 66
tablet_pwr . . . 67
tablet_sar . . . 68
total_dose . . . 69
trunc_lognormal_mean . . . 70
trunc_lognormal_meanlog . . . 70
vr_dose . . . 71
wifi_dose . . . 72
wifi_msar . . . 73
wifi_pwr . . . 74
wifi_sar . . . 74

Index **76**

`RFDose-package` *RFDose: Deterministic and Stochastic RF-EMF Dose Calculator*

Description

Calculates the daily dose of radiofrequency electromagnetic fields (RF-EMF) absorbed by the brain or the whole body, in mJ/kg/day, for each person of a cohort. The doses are calculated from the self-reported use of wireless devices and the environment of each person, for eight exposure sources: mobile phone calls, mobile data, cordless phone, far-field sources, WiFi router, laptop, tablet and other devices. Use `calculate_emf_doses()` for the doses and `calculate_emf_doses_stochastic()` for their uncertainty.

Main changes in version 2.0.0

- SAR values from the GOLIAT dosimetric simulations replace the ETAIN SAR values: one SAR file per phantom (Duke, Ella, Eartha, Thelonious) and one with their average, which is the default. Smartwatch, headphones and Bluetooth headset keep their ETAIN values, as GOLIAT has no similar simulation. The argument `sar_file` chooses the SAR values by name (see `available_sar_files()`), e.g. `sar_file = "goliat_average"`. The ETAIN SAR values of version 1.0.0 remain available.
- Output power parameters updated with data from GOLIAT Task 1.2: 4G and 5G output powers and duty cycles of the mobile phone. The parameters and default values are taken from one Excel file that documents the source of each value; the parameters of version 1.0.0 are kept in `params_v1.0.yaml`. The default value of `use_5g` is now TRUE.
- First version of the stochastic dose model, `calculate_emf_doses_stochastic()`.

Stochastic dose model

`calculate_emf_doses_stochastic()` estimates the uncertainty of the doses. It repeats the dose calculation `n` times (default: 1000). In each draw, three kinds of values vary around the values of the deterministic calculation:

- Parameters: output powers, duty cycles, proportions and far-field power densities, with the distributions of `params_distributions.yaml` (lognormal or truncated lognormal for powers, beta for duty cycles and single proportions, Dirichlet for proportions that sum to 1). The mean of each distribution is the value of the parameter in `params.yaml`. See `simulate_params()`.
- SAR values: lognormal around their value, with a relative standard deviation of 25% (argument `sar_sd`). SAR values from the same simulation are drawn together, and SAR values of 0 stay 0.
- Inputs: durations, proportions, urbanicity and commuting time, with the distributions of `input_distributions.ya` (truncated lognormal with a relative standard deviation of 20% for durations, beta or Dirichlet for proportions, categorical for urbanicity). Each input is drawn around its value in the data, or around its default value if missing. Durations of 0 stay 0. See `simulate_inputs()`; `draw_inputs = FALSE` keeps the inputs fixed.

All rows are calculated with the same parameter draws, and the inputs of a row are drawn around its own values, so the doses of a row do not depend on the other rows. For each row and exposure source, and for the total dose, the result gives the deterministic dose, the mean and standard deviation of the doses of all draws, and their quantiles (default: median and 95% interval). Use `set.seed()` before the calculation for reproducible results.

Author(s)

Maintainer: Irina Wipf <irina.wipf@swisstph.ch>

See Also

`calculate_emf_doses()`, `calculate_emf_doses_stochastic()`, `available_sar_files()`, `load_params()`

act_pwr_props *Get low/lowmed/medhigh/high activity power proportions*

Description

Converts activity durations to proportions.

Usage

```
act_pwr_props(low_dur, lowmed_dur, medhigh_dur, high_dur)
```

Arguments

low_dur	Duration (in seconds per day) of low output power activities
lowmed_dur	Duration (in seconds per day) of low-medium output power activities
medhigh_dur	Duration (in seconds per day) of medium-high output power activities
high_dur	Duration (in seconds per day) of high output power activities

Value

Named list with the use proportion of each activity.

available_sar_files *SAR files included in the package*

Description

Lists the names of the SAR files included in the package. A name can be used instead of a file path to choose the SAR values, e.g. `calculate_emf_doses(data, tissue, sar_file = "goliat_average")`.

Usage

```
available_sar_files()
```

Details

- `"goliat_average"`: GOLIAT, average of the four phantoms (default)
- `"goliat_duke"`, `"goliat_ella"`, `"goliat_eartha"`, `"goliat_thelonious"`: GOLIAT, one phantom
- `"etain"`: ETAIN

Each SAR file also sets the placement proportions that go with its SAR values.

Value

Character vector of SAR file names.

See Also

`load_params()`, `calculate_emf_doses()`

Examples

```
available_sar_files()
```

`calculate_emf_doses` *Calculate Total RF-EMF Doses for ALL samples*

Description

Wrapper function that calculates RF-EMF doses for a data frame of observations. This function will be removed in the future.

Usage

```
calculate_emf_doses(
  data,
  tissue,
  params = NULL,
  default_value_file = NULL,
  sar_file = NULL
)
```

Arguments

`data`	A data frame. Must have the exact same columns as the provided example dataset. Columns must not be missing. Missing values will be replaced with default values.
`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`params`	(experimental) Path to a parameter file in .yaml format. Must match the structure of the internal parameter file. Optional; if not specified, calculations use default parameters. SAR values may be left out (see `sar_file`).
`default_value_file`	List of default values to replace missing values (optional).
`sar_file`	SAR values (optional): the name of a SAR file included in the package, or the path to your own .yaml file with the structure of the included SAR files. Included SAR files (see `available_sar_files()`): • `"goliat_average"`: GOLIAT, average of the four phantoms (default) • `"goliat_duke"`, `"goliat_ella"`, `"goliat_eartha"`, `"goliat_thelonious"`: GOLIAT, one phantom • `"etain"`: ETAIN Its values replace those of `params`, including device parameters it sets with its SAR values (e.g. placement proportions). If not specified, the SAR values of `params` are used, or the default SAR values if `params` has none.

Details

Additional details will follow.

Value

A data frame. Columns contain the calculated RF-EMF dose of each row in mJ/kg/day, for each exposure source and each tissue.

`calculate_emf_doses_stochastic`
Calculate RF-EMF doses with their uncertainty for ALL samples

---

Description

Stochastic version of `calculate_emf_doses()`: calculates the doses of each row many times, each time with other parameter values and input values drawn from their distributions, and summarises the doses of all draws.

Usage

```
calculate_emf_doses_stochastic(
  data,
  tissue,
  n = 1000,
  params = NULL,
  distributions = NULL,
  input_distributions = NULL,
  default_value_file = NULL,
  sar_file = NULL,
  sar_sd = 0.25,
  draw_inputs = TRUE,
  probs = c(0.025, 0.5, 0.975),
  keep_draws = FALSE
)
```

Arguments

`data`	A data frame. Must have the exact same columns as the provided example dataset. Columns must not be missing. Missing values will be replaced with default values.
`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`n`	Number of parameter sets (draws)
`params`	(experimental) Path to a parameter file in .yaml format, as for `calculate_emf_doses()`. Its values are the means of the distributions.
`distributions`	(experimental) Path to a distribution file in .yaml format, with the structure of the internal distribution file (`params_distributions.yaml`). Optional; if not specified, the default distributions are used.
`input_distributions`	(experimental) Path to an input distribution file in .yaml format, with the structure of the internal input distribution file (`input_distributions.yaml`). Optional; if not specified, the default input distributions are used.
`default_value_file`	List of default values to replace missing values (optional).

sar_file	SAR values (optional): the name of a SAR file included in the package (e.g. "goliat_average", the default; see available_sar_files()) or the path to your own .yaml file, as for calculate_emf_doses().
sar_sd	Standard deviation of the SAR values, relative to their value (default: 25%). With 0, the SAR values are not drawn.
draw_inputs	If FALSE, the inputs are not drawn.
probs	Probabilities of the quantiles of the doses (default: median and 95% interval)
keep_draws	If TRUE, also return the doses and parameters of every draw.

Details

The parameter sets are drawn with simulate_params(): parameters with a distribution (see load_distributions()) and the SAR values vary around their value in the parameter and SAR files; all other parameters keep their values. Every row is calculated with the same parameter sets.

The inputs are drawn with simulate_inputs(): inputs with a distribution (see load_input_distributions()) vary around their value in data, or around their default value if missing; all other inputs keep their values.

The model takes the travel time from the time at home. If the travel time of a draw is longer than the drawn time at home, it is capped at the time at home, with a warning.

The doses of a row do not depend on the other rows. Use set.seed() before for reproducible results.

Value

A data frame with one row per row of data and dose, in mJ/kg/day:

- sample_id: ID of the row, if data has a column sample_id
- row: row of data
- dose: exposure source (as the columns of calculate_emf_doses()), or total_dose for their sum
- deterministic: dose with the parameter values of the parameter file, as calculated by calculate_emf_doses()
- mean, sd: mean and standard deviation of the doses of all draws
- one column per quantile of the doses of all draws (e.g. q2.5, q50, q97.5)

With keep_draws = TRUE, a list with this data frame (summary), the doses of every draw (doses, an array row x draw x exposure source), the drawn parameters and SAR values (params, a matrix draw x parameter) and the drawn inputs (inputs, a list of matrices row x draw, one per input).

See Also

calculate_emf_doses(), simulate_params()

check_boolean_not_na *Check if value is logical/Boolean and non-NA*

Description

Check if value is logical/Boolean and non-NA

Usage

```
check_boolean_not_na(x)
```

Arguments

x	Single value to be checked: should be a logical/Boolean, atomic, and not NA to pass check

Value

Returns invisible TRUE if check passes. Side effect of stopping calculation if check fails.

check_character_not_na *Check if value is character and non-NA*

Description

Check if value is character and non-NA

Usage

```
check_character_not_na(x)
```

Arguments

x	Single value to be checked: should be character, atomic, and not NA to pass check

Value

Returns invisible TRUE if check passes. Side effect of stopping calculation if check fails.

check_country *Check if country is valid*

Description

Checks if country code matches a country for which measurement data is available. Use "Other" for any other country.

Usage

```
check_country(country)
```

Arguments

country	Country to check. Must be one of these: "AT" (Austria), "BE" (Belgium), "FR" (France), "HU" (Hungary), "IT" (Italy), "NL" (Netherlands), "PL" (Poland), "ES" (Spain), "CH" (Switzerland), "UK" (United Kingdom), or "Other" (for any other country; calculations use averaged values from countries with available measurements)

Value

Returns invisible TRUE if all checks pass. Side effect of stopping calculations if any check fails.

check_duration *Check input duration(s)*

Description

Checks supplied input duration or vector of durations; needed to stop calculation or raise warning in case of irregular input values.

Usage

```
check_duration(duration)
```

Arguments

duration	single duration (numeric) or vector of durations

Value

Returns invisible TRUE if all checks pass. Side effects of stopping or showing warning message if any check fails

check_headp_num *Check if number of headphones (earbuds) is valid*

Description

Checks if the number of headphones (earbuds) worn during mobile phone calls is either 0, 1, or 2.

Usage

```
check_headp_num(headp_num)
```

Arguments

headp_num	Number of headphones (earbuds) worn during mobile phone call.

Value

Returns invisible TRUE if all checks pass. Side effect of stopping calculations if any check fails.

check_numeric_not_na *Check if value is numeric and non-NA*

Description

Check if value is numeric and non-NA

Usage

```
check_numeric_not_na(x)
```

Arguments

x	Single value to be checked: should be numeric, atomic, and not NA to pass check

Value

Returns invisible TRUE if check passes. Side effect of stopping calculation if check fails.

check_proportions *Check input proportion(s)*

Description

Checks supplied input proportion or vector of proportions; needed to stop calculation or raise warning in case of irregular input values.

Usage

```
check_proportions(proportions)
```

Arguments

proportions	Single proportion value (between 0 and 1) or vector with multiple proportions to check

Value

Returns invisible TRUE if all checks pass. Side effects of stopping or showing warning message if any check fails.

check_shape *Check a shape value of a distribution*

Description

Check a shape value of a distribution

Usage

```
check_shape(x, name)
```

Arguments

x	Value to check: a single number, 0 or more (may be infinite)
name	Name of the value, for the error message

Value

Returns invisible TRUE if the check passes. Side effect of stopping calculation if it fails.

check_tissue *Check if tissue is valid*

Description

Check if tissue with this name can be found in YAML file, and if it has any nested parameters. Parameters themselves are NOT checked for validity.

Usage

```
check_tissue(tissue, device, params)
```

Arguments

tissue	Name of tissue to be checked (character, length 1)
device	Device for which to check tissue (character, length 1)
params	Parameter list, as returned by load_params()

Value

Returns invisible TRUE if all checks pass. Side effect of stopping calculations if any check fails.

check_urbanicity *Check if input urbanicity is valid*

Description

Checks if a supplied urbanicity value is valid (must be "urban", "rural", or "suburban").

Usage

```
check_urbanicity(urbanicity)
```

Arguments

urbanicity	Single urbanicity value (must be one of these: "urban", "rural", "suburban")

Value

Returns invisible TRUE if checks pass. Side effect of stopping calculation if any check fails.

`cordless_dose` *Calculate Dose from Cordless Calls*

Description

Calculates RF-EMF dose from making calls with cordless phone (also known as DECT phone).

Usage

```
cordless_dose(tissue, duration, ear_prop, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration`	Duration of cordless calls in s/day.
`ear_prop`	Proportion of time cordless phone is held against ear during calls.
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The dose is calculated as

$$Dose_{DECT} = mSAR_{DECT} * duration_{DECT}$$

Where:

- $Dose_{DECT}$ is the dose from DECT/cordless phone calls
- $mSAR_{DECT}$ is the momentary SAR value in mJ/kg
- $duration_{DECT}$ is the duration of the calls in s/day

Value

Tissue-specific RF-EMF dose from DECT/cordless phone calls in mJ/kg/day

See Also

`dect_msar()`

Examples

```
cordless_dose(
  tissue      = "brain",
  duration    = 600,
  ear_prop    = 0.9)
```

`dect_msar` *Calculate mSAR from Cordless Calls*

Description

Calculates mSAR (momentary SAR) from making calls with cordless phone (also known as DECT phone).

Usage

```
dect_msar(tissue, ear_prop, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`ear_prop`	Proportion of time cordless phone is held against ear during calls.
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The mSAR is calculated as

$$mSAR_{DECT} = nSAR_{DECT} * outputpower_{DECT}$$

Where:

- $mSAR_{DECT}$ is the mSAR from DECT/cordless phone calls
- $nSAR_{DECT}$ is the nSAR (normalized specific absorption rate) in W/kg/W
- $outputpower_{DECT}$ is the duration of the calls in mW

Value

Tissue-specific mSAR from DECT/cordless phone calls in mW/kg

See Also

`dect_pwr()`, `dect_sar()`

Examples

```
dect_msar(
  tissue      = "brain",
  ear_prop    = 0.9)
```

dect_pwr *Calculate Output Power of DECT/Cordless Phone*

Description

Calculates the output power of DECT/cordless phone during a call.

Usage

```
dect_pwr(params = load_params())
```

Arguments

params	Parameter list (optional). If not specified, calculations use default parameters.

Value

DECT/cordless phone output power in mW

dect_sar *Calculate nSAR from Cordless Calls*

Description

Calculates nSAR (normalized specific absorption rate) from making calls with cordless phone (also known as DECT phone).

Usage

```
dect_sar(tissue, ear_prop, params = load_params())
```

Arguments

tissue	Tissue for which to calculate the nSAR (default: "brain" or "body")
ear_prop	Proportion of time cordless phone is held against ear during calls.
params	Parameter list (optional). If not specified, calculations use default parameters.

Value

Tissue-specific nSAR from DECT/cordless phone calls in W/kg/W

Examples

```
dect_sar(
  tissue      = "brain",
  ear_prop    = 0.9)
```

`example_data` *Example data for testing*

Description

A dataset containing example user data (made up)

Usage

```
example_data
```

Format

A data frame

sample_id test
use_5g test
travel_time test
urbanicity test
country test
headp_ear_num test
mpc_duration test
mpc_ear_prop test
mpc_headp_prop test
mpd_wifi_prop_home test
mpd_wifi_prop_travel test
mpd_wifi_prop_work test
mpd_dur_low test
mpd_dur_lowtomed test
mpd_dur_medtohigh test
mpd_dur_high test
dect_duration test
dect_ear_prop test
lptp_dur_low test
lptp_dur_lowtomed test
lptp_dur_medtohigh test
lptp_dur_high test
tblt_dur_low test
tblt_dur_lowtomed test
tblt_dur_medtohigh test
tblt_dur_high test
hotspot_duration test
smartwatch_duration test
vr_duration test
headphone_duration test
gaming_duration test

Source

Generated for package testing.

`farfield_dose` *Calculate RF-EMF Dose from Far-field Sources*

Description

Calculates the tissue-specific RF-EMF dose from far-field sources

Usage

```
farfield_dose(tissue, country, urbanicity, travel_time, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`country`	Country of residence (Austria = "AT", Belgium = "BE", France = FR, Hungary = "HU", Italy = "IT", Netherlands = "NL", Poland = "PL", Spain = "ES", Switzerland = "CH", United Kingdom = "UK", elsewhere = "Other")
`urbanicity`	Urbanicity of home / workplace (rural, suburban, or urban)
`travel_time`	Time spent commuting in seconds per day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from far-field sources is calculated as:

$$Dose_{farfield} = mSAR_{farfield} * 86400$$

Where:

- $Dose_{farfield}$ is the dose from far-field sources
- $mSAR_{farfield}$ is the momentary SAR value in mJ/kg

Value

Tissue-specific RF-EMF dose from far-field sources in mJ/kg/day

See Also

`farfield_msar()`

Examples

```
farfield_dose(
  tissue      = "body",
  country     = "CH",
  urbanicity  = "suburban",
  travel_time = 1800)
```

`farfield_msar` *Calculate mSAR of Far-field Sources*

Description

Calculates the mSAR of far-field sources

Usage

```
farfield_msar(tissue, country, urbanicity, travel_time, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate nSAR (default: "brain" or "body")
`country`	Country of residence (Austria = "AT", Belgium = "BE", France = FR, Hungary = "HU", Italy = "IT", Netherlands = "NL", Poland = "PL", Spain = "ES", Switzerland = "CH", United Kingdom = "UK", elsewhere = "Other")
`urbanicity`	Urbanicity of home / workplace (rural, suburban, or urban)
`travel_time`	Time spent commuting in seconds per day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the country, urbanicity, and time spent commuting.

Value

mSAR in W/kg/W/m**2

See Also

`farfield_pwr()`, `farfield_sar()`

Examples

```
farfield_msar(
  tissue      = "body",
  country     = "CH",
  urbanicity  = "suburban",
  travel_time = 1800)
```

farfield_pwr *Calculate Output Power of Far-field Sources*

Description

Calculates the output power of far-field sources

Usage

```
farfield_pwr(country, urbanicity, travel_time, params = load_params())
```

Arguments

country	Country of residence (Austria = "AT", Belgium = "BE", France = FR, Hungary = "HU", Italy = "IT", Netherlands = "NL", Poland = "PL", Spain = "ES", Switzerland = "CH", United Kingdom = "UK", elsewhere = "Other")
urbanicity	Urbanicity of home / workplace (rural, suburban, or urban)
travel_time	Time spent commuting in seconds per day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the country, urbanicity, and time spent commuting.

Value

Output power in mW/m**2

Examples

```
farfield_pwr(
country     = "CH",
urbanicity  = "suburban",
travel_time = 1800)
```

farfield_sar *Calculate nSAR from Far-Field Sources*

Description

Calculates tissue-specific nSAR from far-field sources.

Usage

```
farfield_sar(tissue, params = load_params())
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue.

Value

nSAR in W/kg/W/m**2

Examples

```
farfield_sar(
tissue       = "brain")
```

---

fill_missing_variables

Fill missing values in input data frame with missing values

Description

Replaces missing values in input data frame with values specified as default values. In case a column is completely missing, it is added to the data frame, with all values being the default value.

Usage

```
fill_missing_variables(data, defaults, warn_threshold = 0.1)
```

Arguments

data	Input data frame
defaults	Named list (key = column name, value = default value to replace missing values in this column)
warn_threshold	threshold proportion of missing data per column to raise warning

Value

data frame with missing data replaced with default values. Side effect of raising warning if proportion of missing data exceeds warn_threshold for at least one column.

gaming_dose *Calculate RF-EMF Dose from online gaming (console)*

Description

Calculates the tissue-specific RF-EMF dose from online gaming

Usage

```
gaming_dose(tissue, duration_gaming, params = load_params())
```

Arguments

tissue	Tissue for which to calculate dose (default: "brain" or "body")
duration_gaming	duration of gaming with portable console in s/day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from online gaming is calculated as:

$$Dose_{gaming} = mSAR_{gaming} * duration_{gaming}$$

Where:

- $mSAR_{gaming}$ is the mSAR of gaming
- $Dduration_{gaming}$ is the duration of gaming with portable console in s/day

Value

Tissue-specific RF-EMF dose from gaming in mJ/kg/day

Examples

```
gaming_dose(
  tissue          = "body",
  duration_gaming = 600)
```

get_param *Get a parameter value*

Description

Get a parameter value

Usage

```
get_param(params, key)
```

Arguments

params	Parameter list, as returned by load_params()
key	Path of the parameter in the parameter list (character vector)

Value

Value of the parameter. Side effect of stopping calculation if the parameter is not a single number.

headphones_dose *Calculate RF-EMF Dose from Bluetooth Headphones*

Description

Calculates the tissue-specific RF-EMF dose from bluetooth headphones (calls excluded)

Usage

```
headphones_dose(tissue, duration_headphones, params = load_params())
```

Arguments

tissue	Tissue for which to calculate dose (default: "brain" or "body")
duration_headphones	duration of time Bluetooth headphones were used in s/day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from Bluetooth headphone use is calculated as:

$$Dose_{headphones} = Dose_{earset} + Dose_{phone}$$

Where:

- $Dose_{earset}$ is the dose contribution from the headphones themselves
- $Dose_{phone}$ is the dose contribution from the phone connection to the headphones

Value

Tissue-specific RF-EMF dose from Bluetooth headphone use in mJ/kg/day

See Also

`headphones_earset_dose()`, `headphones_phone_dose()`

Examples

```
headphones_dose(
  tissue              = "body",
  duration_headphones = 6000)
```

`headphones_earset_dose` *Calculate RF-EMF Dose from Smart Watch (earset contribution only)*

Description

Calculates the tissue-specific RF-EMF dose from Bluetooth headphone use (earset contribution only).

Usage

```
headphones_earset_dose(tissue, duration_headphones, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration_headphones`	duration of time Bluetooth headphones were worn in s/day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from Bluetooth headphone use (earset contribution) is calculated as:

$$Dose_{earset} = mSAR_{earset} * duration_{earset}$$

Where:

- $mSAR_{earset}$ is the momentary SAR for the headphone connection
- $duration_{earset}$ is the use duration in s/day

Value

Tissue-specific RF-EMF dose from Bluetooth headphone use (earset contribution only) in mJ/kg/day

Examples

```
headphones_earset_dose(
  tissue              = "body",
  duration_headphones = 6000)
```

---

headphones_phone_dose *Calculate RF-EMF Dose from Smart Watch (phone contribution only)*

Description

Calculates the tissue-specific RF-EMF dose from Bluetooth headphone use (phone contribution only).

Usage

```
headphones_phone_dose(tissue, duration_headphones, params = load_params())
```

Arguments

tissue	Tissue for which to calculate dose (default: "brain" or "body")
duration_headphones	duration of time Bluetooth headphones were worn in s/day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from Bluetooth headphone use (phone contribution) is calculated as:

$$Dose_{phone} = mSAR_{phone} * duration_{phone}$$

Where:

- $mSAR_{phone}$ is the momentary SAR for the headphone connection
- $duration_{phone}$ is the use duration in s/day

Value

Tissue-specific RF-EMF dose from Bluetooth headphone use (phone contribution only) in mJ/kg/day

Examples

```
headphones_phone_dose(
  tissue              = "body",
  duration_headphones = 6000)
```

`hotspot_dose` *Calculate RF-EMF Dose from Mobile Phone Hotspot*

Description

Calculates the tissue-specific RF-EMF dose from hotspot

Usage

```
hotspot_dose(tissue, duration_hotspot, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration_hotspot`	duration of time hotspot was active in s/day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from hotspot use is calculated as:

$$Dose_{hotspot} = mSAR_{hotspot} * duration_{hotspot}$$

Where:

- $mSAR_{hotspot}$ is the mSAR of hotspot use
- $Dduration_{hotspot}$ is the duration of hotspot use in s/day

Value

Tissue-specific RF-EMF dose from hotspot use in mJ/kg/day

Examples

```
hotspot_dose(
  tissue           = "body",
  duration_hotspot = 600)
```

`laptop_dose` *Calculate RF-EMF Dose from Laptop Use*

Description

Calculates the tissue-specific RF-EMF dose from laptop use.

Usage

```
laptop_dose(
  tissue,
  dur_low,
  dur_lowmed,
  dur_medhigh,
  dur_high,
  params = load_params()
)
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`dur_low`	Duration (in seconds per day) of low output power activities on laptop
`dur_lowmed`	Duration (in seconds per day) of low-medium output power activities on laptop
`dur_medhigh`	Duration (in seconds per day) of medium-high output power activities on laptop
`dur_high`	Duration (in seconds per day) of high output power activities on laptop
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The laptop RF-EMF dose is calculated as:

$$Dose_{laptop} = mSAR_{laptop} * duration_{laptop}$$

Where:

- $Dose_{laptop}$ is the dose from laptop use
- $mSAR_{laptop}$ is the momentary SAR value in mJ/kg
- $duration_{laptop}$ is the duration of laptop use

We distinguish between 4 types of activities:

1. Low output power: sending e-mails, browsing the internet, scrolling and chatting on social media, sending text messages
2. Low to medium output power: online gaming, streaming music, sending voice messages
3. Medium to high output power: watching videos, uploading pictures or videos, making video calls
4. High output power: uploading large files

Value

Tissue-specific RF-EMF dose from laptop use in mJ/kg/day

See Also

`laptop_msar()`

Examples

```
laptop_dose(
  tissue      = "brain",
  dur_low     = 0,
  dur_lowmed  = 1230,
  dur_medhigh = 0,
  dur_high    = 0)
```

`laptop_msar` *Calculate mSAR from Laptop Use*

Description

Calculates the tissue-specific momentary SAR (mSAR) from laptop use.

Usage

```
laptop_msar(
  tissue,
  dur_low,
  dur_lowmed,
  dur_medhigh,
  dur_high,
  params = load_params()
)
```

Arguments

`tissue`	Tissue for which to calculate mSAR (default: "brain" or "body")
`dur_low`	Duration (in seconds per day) of low output power activities on laptop
`dur_lowmed`	Duration (in seconds per day) of low-medium output power activities on laptop
`dur_medhigh`	Duration (in seconds per day) of medium-high output power activities on laptop
`dur_high`	Duration (in seconds per day) of high output power activities on laptop
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The mSAR is calculated as:

$$mSAR_{laptop} = nSAR_{laptop} * outputpower_{laptop}$$

Where:

- $mSAR_{laptop}$ is the mSAR from laptop use
- $nSAR_{laptop}$ is the normalised SAR value in mJ/kg
- $outputpower_{laptop}$ is the duration of laptop use

We distinguish between 4 types of activities:

1. Low output power: sending e-mails, browsing the internet, scrolling and chatting on social media, sending text messages
2. Low to medium output power: online gaming, streaming music, sending voice messages
3. Medium to high output power: watching videos, uploading pictures or videos, making video calls
4. High output power: uploading large files

Value

Tissue-specific mSAR from laptop use in mW/kg

See Also

`laptop_pwr()`, `laptop_sar()`

Examples

```
laptop_msar(
  tissue      = "brain",
  dur_low     = 0,
  dur_lowmed  = 1230,
  dur_medhigh = 0,
  dur_high    = 0)
```

---

laptop_pwr *Calculate Laptop Output Power*

Description

Calculates the laptop output power during use

Usage

```
laptop_pwr(
  band,
  dur_low,
  dur_lowmed,
  dur_medhigh,
  dur_high,
  params = load_params()
)
```

Arguments

band	WiFi frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
dur_low	Duration (in seconds per day) of low output power activities on laptop
dur_lowmed	Duration (in seconds per day) of low-medium output power activities on laptop
dur_medhigh	Duration (in seconds per day) of medium-high output power activities on laptop
dur_high	Duration (in seconds per day) of high output power activities on laptop
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the frequency band (2.4 GHz or 5.0 GHz) and the type of activity.

Value

Output power in mW

Examples

```
laptop_pwr(
  band        = "2",
  dur_low     = 0,
  dur_lowmed  = 1230,
  dur_medhigh = 0,
  dur_high    = 0)
```

---

laptop_sar *Calculate nSAR during Laptop Use*

Description

Calculates tissue-specific nSAR from laptop use.

Usage

```
laptop_sar(tissue, band, params = load_params())
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue and the frequency band.

Value

nSAR in W/kg/W

Examples

```
laptop_sar(
tissue       = "brain",
band         = "2")
```

---

list_distributions *List parameter distributions*

Description

Turns the nested list of distributions into a flat list, with one element per drawn parameter.

Usage

```
list_distributions(distributions)
```

Arguments

distributions	Nested list of distributions, as returned by load_distributions()

Value

List named by the path of each parameter in the parameter list (sections separated by "."). Each element is the distribution of the parameter, with its path as character vector (key).

list_sar *List SAR values*

Description

Lists the SAR values of a parameter list (the values in the tissue sections of each device), except those that are 0.

Usage

```
list_sar(params)
```

Arguments

params	Parameter list, as returned by load_params()

Value

Data frame with one row per SAR value that is not 0: its path in the parameter list (name, sections separated by "."), its tissue and its value.

load_distributions *Load parameter distributions*

Description

Loads the distributions of the parameters that are drawn in the stochastic dose calculation and returns them as a nested list, with the structure of the parameter list.

Usage

```
load_distributions(path = NULL)
```

Arguments

path	Path of a distribution file (must be a YAML file). If null, the default distributions are loaded.

Details

Each drawn parameter has a distribution and the values that set its shape:

- lognormal: sd_rel (standard deviation relative to the mean)
- trunc_lognormal: sd_rel, min, max (lognormal truncated to min-max)
- beta: alpha (a0 = alpha/mean)
- dirichlet: a0 and group. The parameters of a group are drawn together and sum to 1.

The mean of each distribution is the value of the parameter in the parameter list (see load_params()). Parameters without a distribution are not drawn. The default distributions (params_distributions.yaml) are generated from the same Excel file as the default parameters.

Value

Nested list of distributions.

See Also

simulate_params()

`load_input_distributions` *Load input distributions*

Description

Loads the distributions of the input variables that are drawn in the stochastic dose calculation.

Usage

```
load_input_distributions(path = NULL)
```

Arguments

`path`	Path of an input distribution file (must be a YAML file). If null, the default input distributions are loaded.

Details

Each input is drawn around its value: the value given in the input data, or else its default value. Its distribution only gives the shape:

- `trunc_lognormal`: `sd_rel` (standard deviation relative to the value), `min`, `max` (lognormal truncated to `min`-`max`)
- `lognormal`: `sd_rel`
- `beta`: `alpha` (a0 = alpha/value)
- `dirichlet`: `a0` and `group`. The inputs of a group are shares of a whole and are drawn together with its rest (e.g. the speaker mode of calls).
- `categorical`: `probs`, the probability of each value, for each given value

Inputs without a distribution are not drawn. The default input distributions (`input_distributions.yaml`) are generated from the same Excel file as the default values.

Value

Named list of input distributions.

See Also

simulate_inputs()

`load_params` *Load dose calculation parameters*

Description

Loads default parameters or parameters from external source and returns them as a nested list.

Usage

```
load_params(path = NULL, sar_path = NULL)
```

Arguments

`path`	Path of parameter file to load (must be a YAML file). If null, default non-SAR parameters are loaded. The file may contain SAR values or not.
`sar_path`	SAR values (optional): the name of a SAR file included in the package (see `available_sar_files()`: `"goliat_average"`, `"goliat_duke"`, `"goliat_ella"`, `"goliat_eartha"`, `"goliat_thelonious"` or `"etain"`), or the path of a YAML file with SAR values, with the structure devices > device > tissue > SAR values. Its values replace those of the parameters loaded from `path`. Besides SAR values, it may set device parameters that go with them (e.g. placement proportions such as `legs_prop`).

Details

Default parameters are split in two files: non-SAR parameters (`params.yaml`) and SAR values, which are merged into a single list. The default SAR values are those of GOLIAT, averaged over four phantoms (`"goliat_average"`). The ETAIN SAR values are `"etain"`.

SAR values are taken from `sar_path` if given. Otherwise, the SAR values of the parameter file are used, or the default SAR values if the parameter file has none. A SAR file may also set device parameters that go with its SAR values (e.g. placement proportions), which replace those of the parameter file.

Value

List of parameters loaded from yaml input file(s)

See Also

`load_tissue_params()`, `available_sar_files()`

Examples

```
params <- load_params()
params <- load_params(sar_path = "goliat_average") # the same, with the SAR file named
```

load_tissue_params *Load tissue-specific parameters*

Description

Loads device- and tissue-specific parameters (SAR values) and returns them as a list.

Usage

```
load_tissue_params(params, device_type, tissue_name)
```

Arguments

params	List of parameters (default parameter file or custom YAML that matches default's structure)
device_type	Name of device (default:)
tissue_name	Name of tissue (default: "brain" or "body")

Value

List of parameters (SAR values) specific to selected device type and tissue.

See Also

load_params()

location_props *Calculate proportion of time spent at home vs work vs outdoors vs travelling*

Description

Based on travel time (input variable by user). Assumes a fixed ratio of time spent at home vs at work vs outdoors.

Usage

```
location_props(travel_time, home_prop, work_prop, outd_prop)
```

Arguments

travel_time	daily time spent travelling / commuting in seconds (s)
home_prop	proportion of time spent at home WITHOUT considering commute
work_prop	proportion of time spent at work WITHOUT considering commute
outd_prop	proportion of time spent outside WITHOUT considering commute

Details

We assume that the time spent commuting takes away from time spent at home, but not the time spent at work or outdoors.

Value

List with following elements: "travel", "home", "work", "out", corresponding value is the proportion of time spent in each location.

---

log_pnorm_between *Log of a standard normal probability between two values*

Description

Log of a standard normal probability between two values

Usage

```
log_pnorm_between(a, b)
```

Arguments

a	Lower value
b	Upper value

Value

log(P(a < Z < b)) for a standard normal Z, precise also far in the tails.

---

mobilecall_dose *Calculation of RF-EMF Dose from Mobile Phone Calls*

Description

Includes RF-EMF exposure from phone itself, for Bluetooth connection to headphones, and from headphones themselves

Usage

```
mobilecall_dose(
  tissue,
  duration,
  ear_prop,
  headp_prop,
  urbanicity,
  use_5g,
  travel_time,
  headp_ear_num,
  wifi_prop_home,
  wifi_prop_work,
  wifi_prop_travel,
  params = load_params()
)
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration`	Duration of mobile phone call in seconds per day
`ear_prop`	Proportion of call performed with phone held against ear
`headp_prop`	Proportion of call performed with active Bluetooth connection to headphones
`urbanicity`	Urbanicity of home / workplace (rural, suburban, or urban)
`use_5g`	TRUE if 5g services are used for calls, FALSE if not
`travel_time`	Time spent commuting in seconds per day
`headp_ear_num`	Number of Bluetooth headphones used during call
`wifi_prop_home`	Proportion of time connected to WiFi (vs mobile data) at home
`wifi_prop_work`	Proportion of time connected to WiFi (vs mobile data) at school / work
`wifi_prop_travel`	Proportion of time connected to WiFi (vs mobile data) while commuting
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The mobile call RF-EMF dose is calculated as:

$$Dose_{mobilecall} = Dose_{phone} + Dose_{bluetooth}$$

Where:

- $Dose_{mobilecall}$ is the total dose from all mobile call activities
- $Dose_{phone}$ is the dose from the mobile phone (no Bluetooth)
- $Dose_{bluetooth}$ is the dose from using Bluetooth headphones during the call.

For each source (phone and bluetooth), the dose is calculated as:

$$mSAR * duration$$

Where:

- $mSAR$ is the momentary SAR value in mJ/kg
- $duration$ is the call duration in seconds

Value

Tissue-specific RF-EMF dose in mJ/kg/day

See Also

`mpc_msar()`

Examples

```
mobilecall_dose(
tissue           = "brain",
duration         = 400,
ear_prop         = 0.67,
headp_prop       = 0.17,
urbanicity       = "suburban",
use_5g           = TRUE,
travel_time      = 1800,
headp_ear_num    = 2,
wifi_prop_home   = 1,
wifi_prop_work   = 0.8,
wifi_prop_travel = 0)
```

`mobiledata_dose` *Calculation of RF-EMF Dose from Mobile Phone Data Use*

Description

The mobile data RF-EMF dose is calculated as:

$$Dose_{data} = mSAR_{data} * duration_{data}$$

Usage

```
mobiledata_dose(
  tissue,
  duration_low,
  duration_lowmed,
  duration_medhigh,
  duration_high,
  use_5g,
  wifi_prop_home,
  wifi_prop_work,
  wifi_prop_travel,
  urbanicity,
  travel_time,
  params = load_params()
)
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration_low`	Duration (in seconds per day) of low output power activities
`duration_lowmed`	Duration (in seconds per day) of low-medium output power activities
`duration_medhigh`	Duration (in seconds per day) of medium-high output power activities
`duration_high`	Duration (in seconds per day) of high output power activities
`use_5g`	TRUE if 5G services are used for data transfer, FALSE if not

`wifi_prop_home`	Proportion of time connected to WiFi (vs mobile data) at home
`wifi_prop_work`	Proportion of time connected to WiFi (vs mobile data) at school / work
`wifi_prop_travel`	Proportion of time connected to WiFi (vs mobile data) while commuting
`urbanicity`	Urbanicity of home / workplace (rural, suburban, or urban)
`travel_time`	Time spent commuting in seconds per day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

Where:

- $Dose_{data}$ is the dose from mobile data use
- $mSAR_{data}$ is the momentary SAR value in mJ/kg
- $duration_{data}$ is the duration of mobile data use

We distinguish between 4 types of mobile data use:

1. Low output power: sending e-mails, browsing the internet, scrolling and chatting on social media, sending text messages
2. Low to medium output power: online gaming, streaming music, sending voice messages
3. Medium to high output power: watching videos, uploading pictures or videos, making video calls
4. High output power: uploading large files

Value

Tissue-specific RF-EMF dose from data use in mJ/kg/day

See Also

`mpd_msar()`

Examples

```
mobiledata_dose(
tissue          = "brain",
duration_low    = 4860,
duration_lowmed  = 540,
duration_medhigh = 4860,
duration_high    = 540,
use_5g          = FALSE,
wifi_prop_home   = 0.5,
wifi_prop_work   = 0.5,
wifi_prop_travel = 0.5,
urbanicity      = "suburban",
travel_time     = 1800)
```

mpc_bt_earset_msar *Calculate mobile call mSAR for Bluetooth headphones (headphones only)*

Description

Calculates the mSAR for mobile calling using Bluetooth headphones (headphone/earset only; the contribution of the mobile phone is calculated separately).

Usage

```
mpc_bt_earset_msar(tissue, headp_ear_num, params = load_params())
```

Arguments

tissue	Tissue for which to calculate mSAR (default: "brain" or "body")
headp_ear_num	Number of Bluetooth earphones worn during call (0, 1, or 2)
params	Parameter list (optional). If not specified, calculations use default parameters.

Value

mSAR from Bluetooth calls (headphones/earset only) in mJ/kg/day)

See Also

mpc_bt_phone_msar()

Examples

```
mpc_bt_earset_msar(
  tissue = "brain"
)
```

mpc_bt_phone_msar *Calculate mobile call dose from Bluetooth headphones (phone only)*

Description

Calculates the RF-EMF dose from mobile calling using Bluetooth headphones (phone contribution only; the contribution of the headphones/earset is calculated separately).

Usage

```
mpc_bt_phone_msar(tissue, headp_ear_num, params = load_params())
```

Arguments

tissue	Tissue for which to calculate mSAR (default: "brain" or "body")
headp_ear_num	Number of Bluetooth earphones worn during call (0, 1, or 2)
params	Parameter list (optional). If not specified, calculations use default parameters.

Value

Dose from Bluetooth calls (phone contribution only) in mW/kg)

See Also

mpc_bt_earset_msar()

Examples

```
mpc_bt_phone_msar(
tissue = "brain"
)
```

---

mpc_msar *Calculation of mSAR from Mobile Phone Calls*

Description

Calculates mSAR (momentary specifc absorption rate) from mobile calls for a specific tissue.

Usage

```
mpc_msar(
  tissue,
  prop_native,
  prop_data,
  prop_wifi,
  headp_prop,
  ear_prop,
  speaker_prop,
  urbanicity,
  use_5g,
  travel_time,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate mSAR (default: "brain" or "body")
prop_native	Proportion of mobile call using native connection
prop_data	Proportion of mobile call using mobile data connection
prop_wifi	Proportion of mobile call using WiFi connection
headp_prop	Proportion of call performed with active Bluetooth connection to headphones
ear_prop	Proportion of call performed with phone held against ear
speaker_prop	Proportion of call performed in speaker mode
urbanicity	Urbanicity of home / workplace (rural, suburban, or urban)
use_5g	TRUE if 5g services are used for calls, FALSE if not
travel_time	Time spent commuting in seconds per day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The mSAR is calculated as

$$mSAR = nSAR_{native} \times outputpower_{native} + nSAR_{data} \times outputpower_{data} + nSAR_{wifi} \times outputpower_{wifi}$$

where

- $nSAR_{native}$, $nSAR_{data}$, $nSAR_{wifi}$ are the nSAR (normalized specific absorption rates, in W/kg/W) from mobile phone calls using native, mobile data, or WiFi networks, respectively
- $outputpower_{native}$, $outputpower_{data}$, $outputpower_{wifi}$ are the output powers in mW of the device (mobile phone) using native, mobile data, or WiFi networks, respectively

Value

Momentary SAR (mSAR) in mW/kg

See Also

mpc_pwr_native(), mpc_pwr_data(), mpc_pwr_wifi(), mpc_sar_native(), mpc_sar_data(), mpc_sar_wifi()

Examples

```
mpc_msar(
tissue          = "body",
prop_native     = 0.7,
prop_data       = 0.2,
prop_wifi       = 0.1,
headp_prop      = 0.17,
ear_prop        = 0.67,
speaker_prop    = 0.17,
urbanicity      = "suburban",
use_5g          = TRUE,
travel_time     = 1800)
```

---

mpc_pwr_data *Calculate Mobile Phone Output Power during Data Mobile Phone Calls*

Description

Calculates the mobile phone output power during calls using mobile data.

Usage

```
mpc_pwr_data(band, urbanicity, travel_time, params = load_params())
```

Arguments

band	Technology (3g, 4g, or 5g) used for the call
urbanicity	Urbanicity of home / workplace (rural, suburban, or urban)
travel_time	Time spent commuting in seconds per day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends in the technology used and the location in which the call is performed (urbanicity, indoors/outdoors/commuting)

Value

Output power in mW

Examples

```
mpc_pwr_data(
band        = "4g",
urbanicity  = "suburban",
travel_time = 1800)
```

---

mpc_pwr_native *Calculate Mobile Phone Output Power during Native Mobile Phone Calls*

Description

Calculates the mobile phone output power during calls using native network.

Usage

```
mpc_pwr_native(band, urbanicity, travel_time, params = load_params())
```

Arguments

band	Technology (2G, 3g, 4g, or 5g) used for the call
urbanicity	Urbanicity of home / workplace (rural, suburban, or urban)
travel_time	Time spent commuting in seconds per day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends in the technology used and the location in which the call is performed (urbanicity, indoors/outdoors/commuting)

Value

Output power in mW

Examples

```
mpc_pwr_native(
band        = "4g",
urbanicity  = "suburban",
travel_time = 1800)
```

`mpc_pwr_wifi` *Calculate Mobile Phone Output Power during WiFi Mobile Phone Calls*

Description

Calculates the mobile phone output power during calls using WiFi connection.

Usage

```
mpc_pwr_wifi(band, params = load_params())
```

Arguments

band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz) used for the call
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the frequency band (2.4 GHz or 5.0 GHz)

Value

Output power in mW

Examples

```
mpc_pwr_wifi(
band  = "5")
```

`mpc_sar_data` *Calculate nSAR during Data Mobile Phone Calls*

Description

Calculates tissue-specific nSAR from mobile phone calls using mobile data.

Usage

```
mpc_sar_data(
  tissue,
  band,
  headp_prop,
  ear_prop,
  speaker_prop,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
band	Technology (2G, 3g, 4g, or 5g) used for the call
headp_prop	Proportion of call performed with active Bluetooth connection to headphones
ear_prop	Proportion of call performed with phone held against ear
speaker_prop	Proportion of call performed in speaker mode
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue, the technology (2G, 3g, 4g or 5g), and the location of the phone during the call (against ear, with Bluetooth headphones, in speaker mode)

Value

nSAR in W/kg/W

Examples

```
mpc_sar_data(
tissue       = "brain",
band         = "4g",
headp_prop   = 0.17,
ear_prop     = 0.67,
speaker_prop = 0.17)
```

---

mpc_sar_native *Calculate nSAR during Native Mobile Phone Calls*

Description

Calculates tissue-specific nSAR from mobile phone calls using native network.

Usage

```
mpc_sar_native(
  tissue,
  band,
  headp_prop,
  ear_prop,
  speaker_prop,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
band	Technology (2G, 3g, 4g, or 5g) used for the call
headp_prop	Proportion of call performed with active Bluetooth connection to headphones
ear_prop	Proportion of call performed with phone held against ear
speaker_prop	Proportion of call performed in speaker mode
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue, the technology (2G, 3g, 4g or 5g), and the location of the phone during the call (against ear, with Bluetooth headphones, in speaker mode)

Value

nSAR in W/kg/W

Examples

```
mpc_sar_native(
tissue       = "brain",
band         = "4g",
headp_prop   = 0.17,
ear_prop     = 0.67,
speaker_prop = 0.17)
```

---

mpc_sar_wifi *Calculate nSAR during WiFi Mobile Phone Calls*

Description

Calculates tissue-specific nSAR from mobile phone calls using WiFi connection.

Usage

```
mpc_sar_wifi(
  tissue,
  band,
  headp_prop,
  ear_prop,
  speaker_prop,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz) used for the call
headp_prop	Proportion of call performed with active Bluetooth connection to headphones
ear_prop	Proportion of call performed with phone held against ear
speaker_prop	Proportion of call performed in speaker mode
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue, the technology (2.4gHz or 5.0GHz), and the location of the phone during the call (against ear, with Bluetooth headphones, in speaker mode)

Value

nSAR in W/kg/W

Examples

```
mpc_sar_wifi(
tissue       = "brain",
band         = "2",
headp_prop   = 0.17,
ear_prop     = 0.67,
speaker_prop = 0.17)
```

---

mpd_msar *Calculation of mSAR from Mobile Phone Data*

Description

Calculates mSAR (momentary specifc absorption rate) from data (mobile data and WiFi) use for a specific tissue

Usage

```
mpd_msar(
  tissue,
  duration_low,
  duration_lowmed,
  duration_medhigh,
  duration_high,
  use_5g,
  wifi_prop_home,
  wifi_prop_work,
  wifi_prop_travel,
  urbanicity,
  travel_time,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate mSAR (default: "brain" or "body")
duration_low	Duration (in seconds per day) of low output power activities
duration_lowmed	Duration (in seconds per day) of low-medium output power activities
duration_medhigh	Duration (in seconds per day) of medium-high output power activities
duration_high	Duration (in seconds per day) of high output power activities
use_5g	TRUE if 5G services are used for data transfer, FALSE if not
wifi_prop_home	Proportion of time connected to WiFi (vs mobile data) at home
wifi_prop_work	Proportion of time connected to WiFi (vs mobile data) at school / work
wifi_prop_travel	Proportion of time connected to WiFi (vs mobile data) while commuting
urbanicity	Urbanicity of home / workplace (rural, suburban, or urban)
travel_time	Time spent commuting in seconds per day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The mSAR is calculated as

$$mSAR = nSAR_{mobiledata} \times outputpower_{mobiledata} + nSAR_{wifi} \times outputpower_{wifi}$$

where

- $nSAR_{data}$, $nSAR_{wifi}$ are the nSAR (normalized specific absorption rates, in W/kg/W) from data use using mobile data or WiFi networks, respectively
- $outputpower_{data}$, $outputpower_{wifi}$ are the output powers (mW) of the device (mobilephone) using mobile data or WiFi networks, respectively

Value

Momentary SAR (mSAR) from mobile data use in mW/kg

See Also

mpd_pwr_data(), mpd_pwr_wifi(), mpd_sar_data(), mpd_sar_wifi()

Examples

```
mpd_msar(
tissue           = "brain",
duration_low     = 4860,
duration_lowmed  = 540,
duration_medhigh = 4860,
duration_high    = 540,
use_5g           = FALSE,
wifi_prop_home   = 0.5,
wifi_prop_work   = 0.5,
wifi_prop_travel = 0.5,
urbanicity       = "suburban",
travel_time      = 1800)
```

---

mpd_pwr_data *Calculate Mobile Phone Output Power during Mobile Data Use*

Description

Calculates the mobile phone output power during mobile data use.

Usage

```
mpd_pwr_data(
  band,
  duration_low,
  duration_lowmed,
  duration_medhigh,
  duration_high,
  urbanicity,
  travel_time,
  params = load_params()
)
```

Arguments

band	Technology (3g, 4g, or 5g)
duration_low	Duration (in seconds per day) of low output power activities
duration_lowmed	Duration (in seconds per day) of low-medium output power activities
duration_medhigh	Duration (in seconds per day) of medium-high output power activities

duration_high	Duration (in seconds per day) of high output power activities
urbanicity	Urbanicity of home / workplace (rural, suburban, or urban)
travel_time	Time spent commuting in seconds per day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends in the technology used, the location in which the mobile data network (urbanicity, indoors/outdoors/commuting), and the activity the phone is used for.

Value

Output power in mW

Examples

```
mpd_pwr_data(
band             = "4g",
duration_low     = 4860,
duration_lowmed  = 540,
duration_medhigh = 4860,
duration_high    = 540,
urbanicity       = "rural",
travel_time      = 1800)
```

---

mpd_pwr_wifi *Calculate Mobile Phone Output Power during WiFi Use*

Description

Calculates the mobile phone output power during WiFi use

Usage

```
mpd_pwr_wifi(
  band,
  duration_low,
  duration_lowmed,
  duration_medhigh,
  duration_high,
  params = load_params()
)
```

Arguments

band	WiFi frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
duration_low	Duration (in seconds per day) of low output power activities
duration_lowmed	Duration (in seconds per day) of low-medium output power activities

duration_medhigh	Duration (in seconds per day) of medium-high output power activities
duration_high	Duration (in seconds per day) of high output power activities
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the frequency band (2.4 GHz or 5.0 GHz) and the type of activity.

Value

Output power in mW

Examples

```
mpd_pwr_wifi(
band             = "2",
duration_low     = 4860,
duration_lowmed  = 540,
duration_medhigh = 4860,
duration_high    = 540)
```

---

mpd_sar_data *Calculate nSAR during mobile data use on mobile phone*

Description

Calculates tissue-specific nSAR from mobile data use on mobile phone.

Usage

```
mpd_sar_data(band, tissue, params = load_params())
```

Arguments

band	Technology ("3g", "4g", or "5g")
tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue and the frequency band.

Value

nSAR in W/kg/W

Examples

```
mpd_sar_data(
tissue       = "brain",
band         = "5g")
```

mpd_sar_wifi *Calculate nSAR during WiFi Use on mobile phone*

Description

Calculates tissue-specific nSAR from WiFi use on mobile phone.

Usage

```
mpd_sar_wifi(band, tissue, params = load_params())
```

Arguments

band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue and the frequency band.

Value

nSAR in W/kg/W

Examples

```
mpd_sar_wifi(
tissue       = "brain",
band         = "2")
```

other_dose_wrapper *Calculate RF-EMF Dose from Other Sources*

Description

Wrapper function that calculates total dose from additional devices and activities, as used in the GOLIAT consortium study dose calculation.

Usage

```
other_dose_wrapper(
  tissue,
  duration_hotspot,
  duration_smartwatch,
  duration_vr,
  duration_headphones,
  duration_gaming,
  params = load_params()
)
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration_hotspot`	Duration of time hotspot was active in s/day
`duration_smartwatch`	Duration of time smartwatch was used in s/day
`duration_vr`	Duration of time VR headset was used in s/day
`duration_headphones`	Duration of time Bluetooth headphones were used in s/day
`duration_gaming`	Duration of gaming with portable console in s/day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from other sources takes into account 5 additional sources:

1. Hotspot (smartphone)
2. Smart watch
3. Virtual reality (VR) headset
4. Bluetooth headphones
5. Portable gaming consoles

The dose is calculated as

$$Dose_{other} = Dose_{hotspot} + Dose_{smartwatch} + Dose_{vr} + Dose_{headphones} + Dose_{gaming}$$

The dose from each source is calculated from the corresponding dose functions.

Value

Tissue-specific RF-EMF dose from other sources in mJ/kg/day

See Also

`hotspot_dose()`, `smartwatch_dose()`, `vr_dose()`, `headphones_dose()`, `gaming_dose()`

Examples

```
other_dose_wrapper(
  tissue             = "brain",
  duration_hotspot   = 600,
  duration_smartwatch = 86400,
  duration_vr        = 0,
  duration_headphones = 8280,
  duration_gaming    = 3600)
```

prepare_dose_data *Prepare input data for the dose calculation*

Description

Replaces missing values in the input data with default values.

Usage

```
prepare_dose_data(data, default_value_file = NULL)
```

Arguments

data	A data frame. Must have the exact same columns as the provided example dataset.
default_value_file	Path to a .yaml file with default values to replace missing values (optional). If not specified, the internal default values are used.

Value

The data frame with missing values replaced. Side effect of raising a warning if a column has more than 10% missing values.

q_around *Quantiles of a lognormal distribution around a value*

Description

Quantiles of a lognormal distribution around a value

Usage

```
q_around(p, value, spec)
```

Arguments

p	Vector of probabilities
value	Value of the input, the mean of the distribution
spec	Distribution: sd_rel, and for a truncated distribution min and max

Value

Vector of quantiles. All quantiles are equal to value if it is 0 or at (or beyond) a bound.

q_beta *Quantiles of a beta distribution*

Description

Quantile function of the beta distribution of `r_beta()`.

Usage

```
q_beta(p, mean, alpha)
```

Arguments

`p`	Vector of probabilities
`mean`	Mean (between 0 and 1)
`alpha`	First shape parameter: the concentration is a0 = alpha/mean

Value

Vector of quantiles between 0 and 1, increasing with `p`. All quantiles are equal to `mean` if `mean` is 0 or 1, or `alpha` is infinite.

q_category *Quantiles of a categorical distribution*

Description

Quantiles of a categorical distribution

Usage

```
q_category(p, value, probs)
```

Arguments

`p`	Vector of probabilities
`value`	Given value of the input
`probs`	Probability of each value, for each given value

Value

Vector of drawn values.

q_shares *Quantiles of shares of a whole*

Description

Dirichlet distribution of shares and the rest of their whole, from the quantiles of independent gamma distributions.

Usage

```
q_shares(p, values, a0)
```

Arguments

p	Matrix of probabilities: one row per draw, one column per share and one for the rest
values	Shares (their sum is 1 at most)
a0	Concentration: the larger, the closer the draws are to values

Value

Matrix with one row per draw and one column per share. Shares of 0 stay 0.

q_trunc_lognormal *Quantiles of a truncated lognormal distribution*

Description

Quantile function of the truncated lognormal distribution of r_trunc_lognormal(), which keeps the given mean.

Usage

```
q_trunc_lognormal(p, mean, sd, min, max)
```

Arguments

p	Vector of probabilities
mean	Mean of the truncated distribution (between min and max)
sd	Standard deviation, before truncation
min	Lower bound (0 or more)
max	Upper bound

Details

The probabilities are computed as logarithms, so that the quantiles are also right very close to a bound (e.g. a mean just below max), where the probabilities can be too small to be stored as numbers.

Value

Vector of quantiles between `min` and `max`, increasing with `p`. All quantiles are equal to `mean` if `sd` is 0.

---

r_beta *Draw from a beta distribution*

Description

Draw from a beta distribution

Usage

```
r_beta(n, mean, alpha)
```

Arguments

`n`	Number of draws
`mean`	Mean (between 0 and 1)
`alpha`	First shape parameter: the concentration is a0 = alpha/mean

Value

Vector of `n` draws between 0 and 1. All draws are equal to `mean` if `mean` is 0 or 1, or `alpha` is infinite.

---

r_dirichlet *Draw from a Dirichlet distribution*

Description

Draw from a Dirichlet distribution

Usage

```
r_dirichlet(n, mean, a0)
```

Arguments

`n`	Number of draws
`mean`	Vector of means (proportions that sum to 1)
`a0`	Concentration: the larger, the closer the draws are to `mean`

Value

Matrix with one row per draw and one column per proportion; each row sums to 1. Proportions with mean 0 stay 0. All rows are equal to `mean` if `a0` is infinite.

r_lognormal *Draw from a lognormal distribution*

Description

Draw from a lognormal distribution

Usage

```
r_lognormal(n, mean, sd)
```

Arguments

n	Number of draws
mean	Mean (arithmetic, not on the log scale)
sd	Standard deviation (arithmetic, not on the log scale)

Value

Vector of n draws. All draws are equal to mean if mean or sd is 0.

r_trunc_lognormal *Draw from a truncated lognormal distribution*

Description

Draws from a lognormal distribution truncated to the interval min-max, which keeps the given mean.

Usage

```
r_trunc_lognormal(n, mean, sd, min, max)
```

Arguments

n	Number of draws
mean	Mean of the truncated distribution (between min and max)
sd	Standard deviation, before truncation
min	Lower bound (0 or more)
max	Upper bound

Details

Truncation alone would move the mean, e.g. down if the mean is close to max. The lognormal distribution before truncation is therefore shifted (on the log scale), until the mean of the truncated distribution is mean. The spread on the log scale is the one of a lognormal distribution with mean and sd; truncation makes the standard deviation of the draws smaller than sd.

Value

Vector of n draws between min and max. All draws are equal to mean if sd is 0.

sar_file_path *Path of a SAR file*

Description

Path of a SAR file

Usage

```
sar_file_path(sar_file)
```

Arguments

sar_file	Name of a SAR file included in the package (see available_sar_files(), case is ignored) or path of a YAML file

Value

Path of the SAR file. Side effect of stopping calculation if the file does not exist.

set_params *Set parameter values*

Description

Replaces the values of some parameters, e.g. by one draw of simulate_params().

Usage

```
set_params(params, values, keys = strsplit(names(values), ".", fixed = TRUE))
```

Arguments

params	Parameter list, as returned by load_params()
values	Numeric vector of new values
keys	List with the path of each parameter of values (character vectors). By default, the names of values, with sections separated by ".".

Value

Parameter list with the new values.

simulate_inputs *Draw input values*

Description

Draws the input values of each row for the stochastic dose calculation, each around its value in the input data.

Usage

```
simulate_inputs(n, data, input_distributions = load_input_distributions())
```

Arguments

n	Number of draws
data	Data frame of inputs without missing values (see calculate_emf_doses())
input_distributions	Named list of input distributions, as returned by load_input_distributions()

Details

All rows are drawn with the same random numbers, each transformed by the distribution around its own value, so the draws of a row do not depend on the other rows. Durations of 0 stay 0, and so do values at the bounds of a truncated distribution. The inputs of a Dirichlet group are drawn together with the rest of their whole, so they sum to 1 at most. Use set.seed() before for reproducible draws.

Value

Named list with one element per drawn input: a matrix with one row per row of data and one column per draw.

See Also

load_input_distributions()

simulate_params *Draw parameter sets*

Description

Draws parameter sets for the stochastic dose calculation: each drawn parameter from its distribution, around its value in the parameter list.

Usage

```
simulate_params(
  n,
  params = load_params(),
  distributions = load_distributions(),
  sar_sd = 0.25
)
```

Arguments

n	Number of parameter sets
params	Parameter list, as returned by load_params(). Its values are the means of the distributions.
distributions	Nested list of distributions, as returned by load_distributions()
sar_sd	Standard deviation of the SAR values, relative to their value. With 0, the SAR values are not drawn.

Details

The parameters of a Dirichlet group are drawn together, so that they sum to 1 in every draw.

The SAR values are drawn too, from a lognormal distribution around their value in the parameter list, with standard deviation sar_sd times the value. SAR values with the same value for the same tissue come from the same simulation and are drawn together; SAR values of 0 stay 0.

The SAR values are drawn after the other parameters, so the draws of the other parameters do not depend on sar_sd. Use set.seed() before for reproducible draws.

Value

Matrix with one row per parameter set and one column per drawn parameter and SAR value, named by its path in the parameter list (sections separated by "."). Parameters that are not drawn keep their value in params.

See Also

load_distributions()

---

smartwatch_dose *Calculate RF-EMF Dose from Smart Watch*

Description

Calculates the tissue-specific RF-EMF dose from smart-watch use

Usage

```
smartwatch_dose(tissue, duration_smartwatch, params = load_params())
```

Arguments

tissue	Tissue for which to calculate dose (default: "brain" or "body")
duration_smartwatch	duration of time smartwatch was worn in s/day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from smart-watch use is calculated as:

$$Dose_{smartwatch} = Dose_{watch} + Dose_{phone}$$

Where:

- $Dose_{watch}$ is the dose contribution from the smart-watch itself
- $Dose_{phone}$ is the dose contribution from the phone connection to the smart-watch

Value

Tissue-specific RF-EMF dose from smart-watch use in mJ/kg/day

See Also

`smartwatch_watch_dose()`, `smartwatch_phone_dose()`

Examples

```
smartwatch_dose(
  tissue              = "body",
  duration_smartwatch = 6000)
```

---

smartwatch_phone_dose *Calculate RF-EMF Dose from Smart Watch (phone contribution only)*

Description

Calculates the tissue-specific RF-EMF dose from smart-watch use (phone contribution only).

Usage

```
smartwatch_phone_dose(tissue, duration_smartwatch, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration_smartwatch`	duration of time smart-watch was worn in s/day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from smart-watch use (phone contribution) is calculated as:

$$Dose_{phone} = mSAR_{phone} * duration_{phone}$$

Where:

- $mSAR_{phone}$ is the momentary SAR
- $duration_{phone}$ is the use duration

Value

Tissue-specific RF-EMF dose from smart-watch use (phone contribution only) in mJ/kg/day

Examples

```
smartwatch_phone_dose(
  tissue              = "body",
  duration_smartwatch = 6000)
```

---

smartwatch_watch_dose *Calculate RF-EMF Dose from Smart Watch (watch contribution only)*

Description

Calculates the tissue-specific RF-EMF dose from smart-watch use (watch contribution only).

Usage

```
smartwatch_watch_dose(tissue, duration_smartwatch, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate dose (default: "brain" or "body")
`duration_smartwatch`	duration of time smart-watch was worn in s/day
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from smart-watch use (watch contribution) is calculated as:

$$Dose_{watch} = mSAR_{watch} * duration_{watch}$$

Where:

- $mSAR_{watch}$ is the momentary SAR
- $duration_{watch}$ is the use duration

While worn, the smart-watch transfers data for `watch_transfer_dur` seconds per day of wearing. During the share `watch_transfer_active_prop` of this time it transmits actively, the rest of the time passively (parameters of the device `smartwatch`).

Value

Tissue-specific RF-EMF dose from smart-watch use (watch contribution only) in mJ/kg/day

Examples

```
smartwatch_watch_dose(
  tissue              = "body",
  duration_smartwatch = 6000)
```

stochastic_doses *Calculate doses for drawn parameter sets*

Description

Calculates the doses of each sample with each parameter set and its drawn inputs.

Usage

```
stochastic_doses(
  samples,
  tissue,
  params,
  draws,
  inputs = NULL,
  known_warnings = character(0)
)
```

Arguments

samples	List of samples (input values of a single observation each), as for total_dose()
tissue	Tissue for which to calculate dose (default: "brain" or "body")
params	Parameter list, as returned by load_params()
draws	Drawn parameter sets, as returned by simulate_params()
inputs	Drawn inputs, as returned by simulate_inputs() (optional). If NULL, the inputs of the samples are used in every draw.
known_warnings	Warning messages that were already raised. Other warnings are raised once, with the number of times they occurred.

Value

Array of doses in mJ/kg/day: sample x draw x exposure source.

summarise_doses *Summarise the doses of all draws*

Description

Summarise the doses of all draws

Usage

```
summarise_doses(deterministic, doses, probs)
```

Arguments

deterministic	Matrix of doses with the parameter values of the parameter file: sample x exposure source
doses	Array of doses with drawn parameter sets: sample x draw x exposure source, as returned by stochastic_doses()
probs	Probabilities of the quantiles

Value

Data frame with one row per sample and dose (each exposure source and their sum, total_dose).

---

tablet_dose *Calculate RF-EMF Dose from Tablet Use*

Description

Calculates the tissue-specific RF-EMF dose from tablet use.

Usage

```
tablet_dose(
  tissue,
  dur_low,
  dur_lowmed,
  dur_medhigh,
  dur_high,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate dose (default: "brain" or "body")
dur_low	Duration (in seconds per day) of low output power activities on tablet
dur_lowmed	Duration (in seconds per day) of low-medium output power activities on tablet
dur_medhigh	Duration (in seconds per day) of medium-high output power activities on tablet
dur_high	Duration (in seconds per day) of high output power activities on tablet
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The tablet RF-EMF dose is calculated as:

$$Dose_{tablet} = mSAR_{tablet} * duration_{tablet}$$

Where:

- $Dose_{tablet}$ is the dose from tablet use
- $mSAR_{tablet}$ is the momentary SAR value in mJ/kg
- $duration_{tablet}$ is the duration of tablet use

We distinguish between 4 types of activities:

1. Low output power: sending e-mails, browsing the internet, scrolling and chatting on social media, sending text messages
2. Low to medium output power: online gaming, streaming music, sending voice messages
3. Medium to high output power: watching videos, uploading pictures or videos, making video calls
4. High output power: uploading large files

Value

Tissue-specific RF-EMF dose from tablet use in mJ/kg/day

See Also

`tablet_msar()`

Examples

```
tablet_dose(
  tissue      = "brain",
  dur_low     = 0,
  dur_lowmed  = 681,
  dur_medhigh = 0,
  dur_high    = 0)
```

---

tablet_msar *Calculate mSAR from Tablet Use*

Description

Calculates the tissue-specific momentary SAR (mSAR) from tablet use.

Usage

```
tablet_msar(
  tissue,
  dur_low,
  dur_lowmed,
  dur_medhigh,
  dur_high,
  params = load_params()
)
```

Arguments

tissue	Tissue for which to calculate mSAR (default: "brain" or "body")
dur_low	Duration (in seconds per day) of low output power activities on tablet
dur_lowmed	Duration (in seconds per day) of low-medium output power activities on tablet
dur_medhigh	Duration (in seconds per day) of medium-high output power activities on tablet
dur_high	Duration (in seconds per day) of high output power activities on tablet
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The mSAR is calculated as:

$$mSAR_{tablet} = nSAR_{tablet} * outputpower_{tablet}$$

Where:

- $mSAR_{tablet}$ is the mSAR from tablet use
- $nSAR_{tablet}$ is the normalised SAR value in mJ/kg
- $outputpower_{tablet}$ is the duration of tablet use

We distinguish between 4 types of activities:

1. Low output power: sending e-mails, browsing the internet, scrolling and chatting on social media, sending text messages
2. Low to medium output power: online gaming, streaming music, sending voice messages
3. Medium to high output power: watching videos, uploading pictures or videos, making video calls
4. High output power: uploading large files

Value

Tissue-specific mSAR from tablet use in mW/kg

See Also

tablet_pwr(), tablet_sar()

Examples

```
tablet_msar(
  tissue      = "brain",
  dur_low     = 0,
  dur_lowmed  = 1230,
  dur_medhigh = 0,
  dur_high    = 0)
```

---

tablet_pwr *Calculate Tablet Output Power*

Description

Calculates the tablet output power during use

Usage

```
tablet_pwr(
  band,
  dur_low,
  dur_lowmed,
  dur_medhigh,
  dur_high,
  params = load_params()
)
```

Arguments

band	WiFi frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
dur_low	Duration (in seconds per day) of low output power activities on tablet
dur_lowmed	Duration (in seconds per day) of low-medium output power activities on tablet
dur_medhigh	Duration (in seconds per day) of medium-high output power activities on tablet
dur_high	Duration (in seconds per day) of high output power activities on tablet
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the frequency band (2.4 GHz or 5.0 GHz) and the type of activity.

Value

Output power in mW

Examples

```
tablet_pwr(
  band        = "5",
  dur_low     = 0,
  dur_lowmed  = 681,
  dur_medhigh = 0,
  dur_high    = 0)
```

---

tablet_sar *Calculate nSAR during Tablet Use*

Description

Calculates tissue-specific nSAR from tablet use.

Usage

```
tablet_sar(tissue, band, params = load_params())
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue and the frequency band.

Value

nSAR in W/kg/W

Examples

```
tablet_sar(
tissue       = "brain",
band         = "2")
```

---

total_dose *Calculate Total RF-EMF Dose from All Sources*

Description

Calculates the total RF-EMF dose from all available RF-EMF sources for a specific tissue.

Usage

```
total_dose(sample, tissue, params = load_params())
```

Arguments

sample	A list with input values for a single observation. List elements must match the names and data types of the provided example dataset and must not be NA.
tissue	Tissue for which to calculate dose (default: "brain" or "body")
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

A detailed description of the dose calculation will follow.

Value

Total RF-EMF dose in mJ/kg/day for the specified tissue.

trunc_lognormal_mean *Mean of a truncated lognormal distribution*

Description

Mean of a truncated lognormal distribution

Usage

```
trunc_lognormal_mean(meanlog, sdlog, min, max)
```

Arguments

meanlog	Mean on the log scale, before truncation
sdlog	Standard deviation on the log scale, before truncation
min	Lower bound
max	Upper bound

Value

Mean of the lognormal distribution truncated to min-max.

trunc_lognormal_meanlog *Log-scale mean of a lognormal distribution, from the mean after truncation*

Description

Log-scale mean of a lognormal distribution, from the mean after truncation

Usage

```
trunc_lognormal_meanlog(mean, sdlog, min, max)
```

Arguments

mean	Mean of the truncated distribution
sdlog	Standard deviation on the log scale, before truncation
min	Lower bound
max	Upper bound

Value

Mean on the log scale, before truncation, for which the lognormal distribution truncated to min-max has the mean mean.

vr_dose *Calculate RF-EMF Dose from Virtual Reality Headset*

Description

Calculates the tissue-specific RF-EMF dose from VR headset

Usage

```
vr_dose(tissue, duration_vr, params = load_params())
```

Arguments

tissue	Tissue for which to calculate dose (default: "brain" or "body")
duration_vr	duration of time VR headset was used in s/day
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from VR headset is calculated as:

$$Dose_{vr} = mSAR_{vr} * duration_{vr}$$

Where:

- $mSAR_{vr}$ is the mSAR of virtual reality headet use
- $Dduration_{vr}$ is the duration of VR headset use in s/day

Value

Tissue-specific RF-EMF dose from VR headset use in mJ/kg/day

Examples

```
vr_dose(
  tissue      = "brain",
  duration_vr = 600)
```

`wifi_dose` *Calculate RF-EMF Dose from WiFi Router*

Description

Calculates the tissue-specific RF-EMF dose from WiFi router

Usage

```
wifi_dose(tissue, travel_time, wifi_prop_travel, params = load_params())
```

Arguments

`tissue`	Tissue for which to calculate nSAR (default: "brain" or "body")
`travel_time`	Time spent commuting in seconds per day
`wifi_prop_travel`	Proportion of time connected to WiFi (vs mobile data) while commuting
`params`	Parameter list (optional). If not specified, calculations use default parameters.

Details

The RF-EMF dose from WiFi router is calculated as:

$$Dose_{WiFi} = mSAR_{farfield} * duration_{WiFi}$$

Where:

- $Dose_{WiFi}$ is the dose from WiFi Router
- $mSAR_{WiFi}$ is the momentary SAR value in mJ/kg
- $duration_{WiFi}$ is the duration of time exposed to a WiFi router in seconds/day

Value

Tissue-specific RF-EMF dose from Wifi router in mJ/kg/day

See Also

`wifi_msar()`

Examples

```
wifi_dose(
  tissue          = "body",
  travel_time     = 1800,
  wifi_prop_travel = 0.5)
```

wifi_msar *Calculate mSAR of WiFi Router*

Description

Calculates the mSAR of WiFi Router

Usage

```
wifi_msar(tissue, travel_time, wifi_prop_travel, params = load_params())
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
travel_time	Time spent commuting in seconds per day
wifi_prop_travel	Proportion of time connected to WiFi (vs mobile data) while commuting
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the time spent commuting and the proportion of time a WiFi connection (vs mobile data) is used while commuting.

Value

mSAR in W/kg/W/m**2

See Also

wifi_pwr(), wifi_sar()

Examples

```
wifi_msar(
  tissue           = "body",
  travel_time      = 1800,
  wifi_prop_travel = 0.5)
```

wifi_pwr *Calculate Output Power of WiFi Router*

Description

Calculates the output power of WiFi router

Usage

```
wifi_pwr(band, params = load_params())
```

Arguments

band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The output power depends on the WiFi frequency band.

Value

Output power in mW/m**2

Examples

```
wifi_pwr(
band = "2")
```

wifi_sar *Calculate nSAR from WiFi Router*

Description

Calculates tissue-specific nSAR from WiFi Router

Usage

```
wifi_sar(tissue, band, params = load_params())
```

Arguments

tissue	Tissue for which to calculate nSAR (default: "brain" or "body")
band	Frequency band ("2" for 2.4 GHz, "5" for 5.0 GHz)
params	Parameter list (optional). If not specified, calculations use default parameters.

Details

The normalized specific absorption rate (nSAR) depends on the tissue and the WiFi frequency band.

Value

nSAR in W/kg/W/m**2

Examples

```
wifi_sar(
tissue = "brain",
band   = "5")
```

Index

∗ **datasets**
    example_data, 17

act_pwr_props, 5
available_sar_files, 5
available_sar_files(), *4*, *6*, *8*, *34*, *59*

calculate_emf_doses, 6
calculate_emf_doses(), *3–5*, *7, 8*, *60*
calculate_emf_doses_stochastic, 7
calculate_emf_doses_stochastic(), *3, 4*
check_boolean_not_na, 9
check_character_not_na, 9
check_country, 10
check_duration, 10
check_headp_num, 11
check_numeric_not_na, 11
check_proportions, 12
check_shape, 12
check_tissue, 13
check_urbanicity, 13
cordless_dose, 14

dect_msar, 15
dect_msar(), *14*
dect_pwr, 16
dect_pwr(), *15*
dect_sar, 16
dect_sar(), *15*

example_data, 17

farfield_dose, 18
farfield_msar, 19
farfield_msar(), *18*
farfield_pwr, 20
farfield_pwr(), *19*
farfield_sar, 20
farfield_sar(), *19*
fill_missing_variables, 21

gaming_dose, 22
gaming_dose(), *53*
get_param, 23

headphones_dose, 23
headphones_dose(), *53*
headphones_earset_dose, 24
headphones_earset_dose(), *24*
headphones_phone_dose, 25
headphones_phone_dose(), *24*
hotspot_dose, 26
hotspot_dose(), *53*

laptop_dose, 27
laptop_msar, 28
laptop_msar(), *28*
laptop_pwr, 29
laptop_pwr(), *29*
laptop_sar, 30
laptop_sar(), *29*
list_distributions, 31
list_sar, 32
load_distributions, 32
load_distributions(), *8*, *31*, *61*
load_input_distributions, 33
load_input_distributions(), *8*, *60*
load_params, 34
load_params(), *4, 5*, *13*, *23*, *32*, *35*, *59*, *61*, *64*
load_tissue_params, 35
load_tissue_params(), *34*
location_props, 35
log_pnorm_between, 36

mobilecall_dose, 36
mobiledata_dose, 38
mpc_bt_earset_msar, 40
mpc_bt_earset_msar(), *41*
mpc_bt_phone_msar, 40
mpc_bt_phone_msar(), *40*
mpc_msar, 41
mpc_msar(), *37*
mpc_pwr_data, 42
mpc_pwr_data(), *42*
mpc_pwr_native, 43
mpc_pwr_native(), *42*
mpc_pwr_wifi, 44
mpc_pwr_wifi(), *42*
mpc_sar_data, 44

mpc_sar_data(), *42*
mpc_sar_native, 45
mpc_sar_native(), *42*
mpc_sar_wifi, 46
mpc_sar_wifi(), *42*
mpd_msar, 47
mpd_msar(), *39*
mpd_pwr_data, 49
mpd_pwr_data(), *49*
mpd_pwr_wifi, 50
mpd_pwr_wifi(), *49*
mpd_sar_data, 51
mpd_sar_data(), *49*
mpd_sar_wifi, 52
mpd_sar_wifi(), *49*

other_dose_wrapper, 52

prepare_dose_data, 54

q_around, 54
q_beta, 55
q_category, 55
q_shares, 56
q_trunc_lognormal, 56

r_beta, 57
r_beta(), *55*
r_dirichlet, 57
r_lognormal, 58
r_trunc_lognormal, 58
r_trunc_lognormal(), *56*
RFDose *(*RFDose-package*)*, 3
RFDose-package, 3

sar_file_path, 59
set.seed(), *4*, *8*, *60, 61*
set_params, 59
simulate_inputs, 60
simulate_inputs(), *4*, *8*, *33*, *64*
simulate_params, 60
simulate_params(), *4*, *8*, *33*, *59*, *64*
smartwatch_dose, 61
smartwatch_dose(), *53*
smartwatch_phone_dose, 62
smartwatch_phone_dose(), *62*
smartwatch_watch_dose, 63
smartwatch_watch_dose(), *62*
stochastic_doses, 64
stochastic_doses(), *65*
summarise_doses, 64

tablet_dose, 65
tablet_msar, 66
tablet_msar(), *66*
tablet_pwr, 67
tablet_pwr(), *67*
tablet_sar, 68
tablet_sar(), *67*
total_dose, 69
total_dose(), *64*
trunc_lognormal_mean, 70
trunc_lognormal_meanlog, 70

vr_dose, 71
vr_dose(), *53*

wifi_dose, 72
wifi_msar, 73
wifi_msar(), *72*
wifi_pwr, 74
wifi_pwr(), *73*
wifi_sar, 74
wifi_sar(), *73*